COMP3230 · PRINCIPLES OF OPERATING SYSTEMS

Tutorial 1 — Warm Up

From OS Concepts to C Programming

Mingyan Gao · 2026
mingyan.gao@connect.hku.hk

What We Will Do Today

PART 01

Lecture Review

- User vs. kernel mode
- Process abstraction
- System calls · switches · interrupts

→

PART 02

Linux Workflow

- Workbench & command line
- GCC · Make · Git
- Build → run → debug → test

→

PART 03

C for OS

- Pointers · arrays · strings
- Structs · memory
- Low-level I/O

→

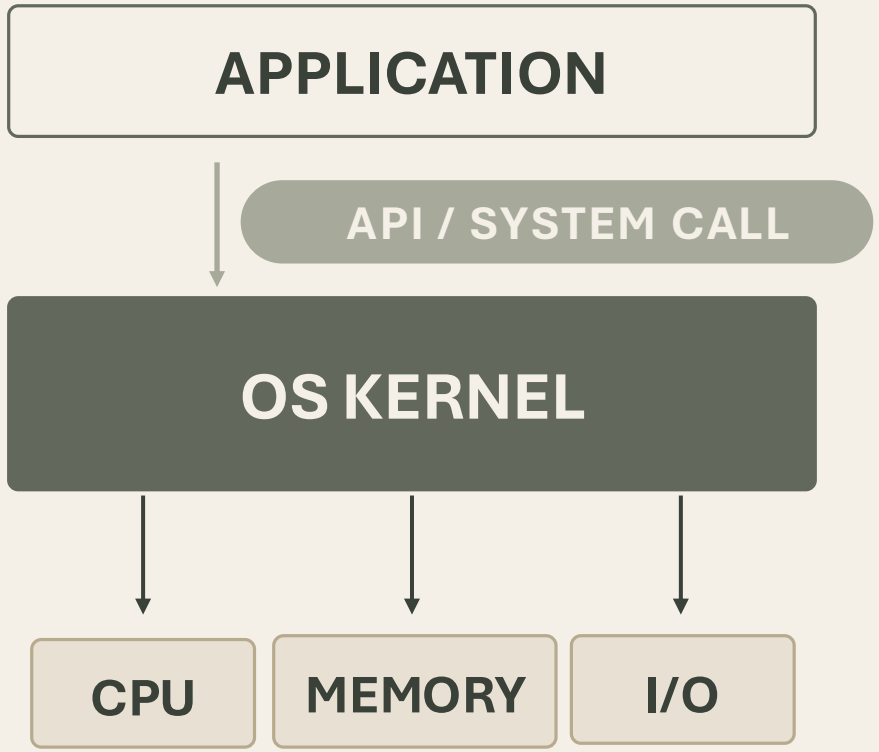
PART 04

Put It Together

- Signals
- Debugging
- Readiness check

Come on guys! Let’s do the work. Hope you enjoy!

The One Picture Behind the First Three Lectures



01 How does an app request an OS service?

SYSTEM CALL

02 How can many processes share one CPU?

CONTEXT SWITCH

03 How can the OS regain CPU control?

INTERRUPT

User Mode vs. Kernel Mode

RESTRICTED

User Mode

- Application code
- Restricted privileges
- No privileged instructions
- No free hardware access
- No access to another process's memory

PRIVILEGED

Kernel Mode

- Operating system kernel
- Highest privilege level
- Executes privileged instructions
- Controls hardware
- Manages protected resources



What Is a Process?

PASSIVE

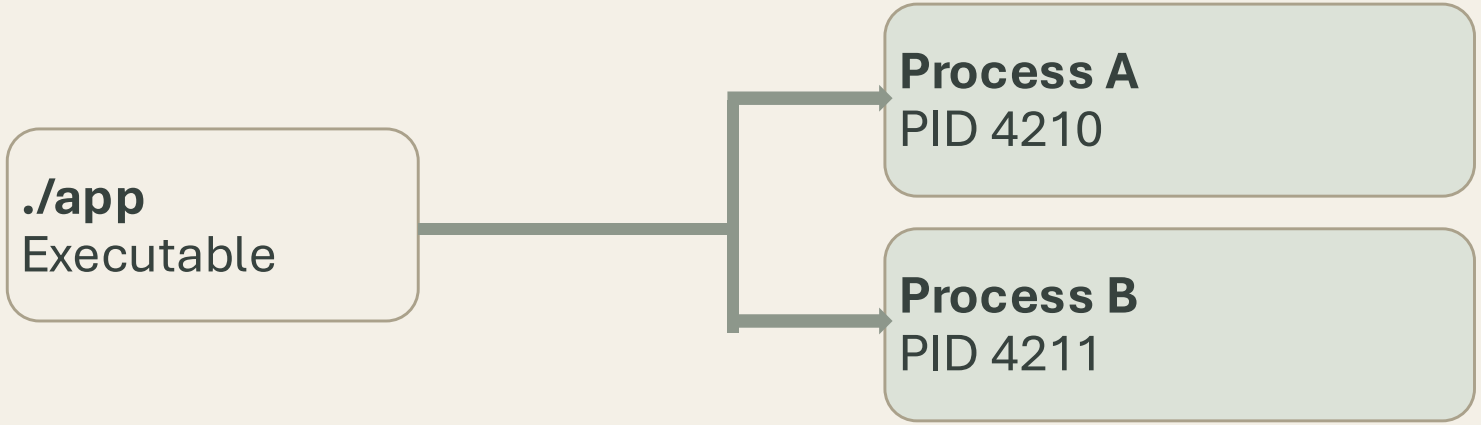
Program

- Executable instructions on disk
- Not currently executing

ACTIVE

Process

- A program in execution
- Has execution state
- Owns resources
- Can be scheduled by OS



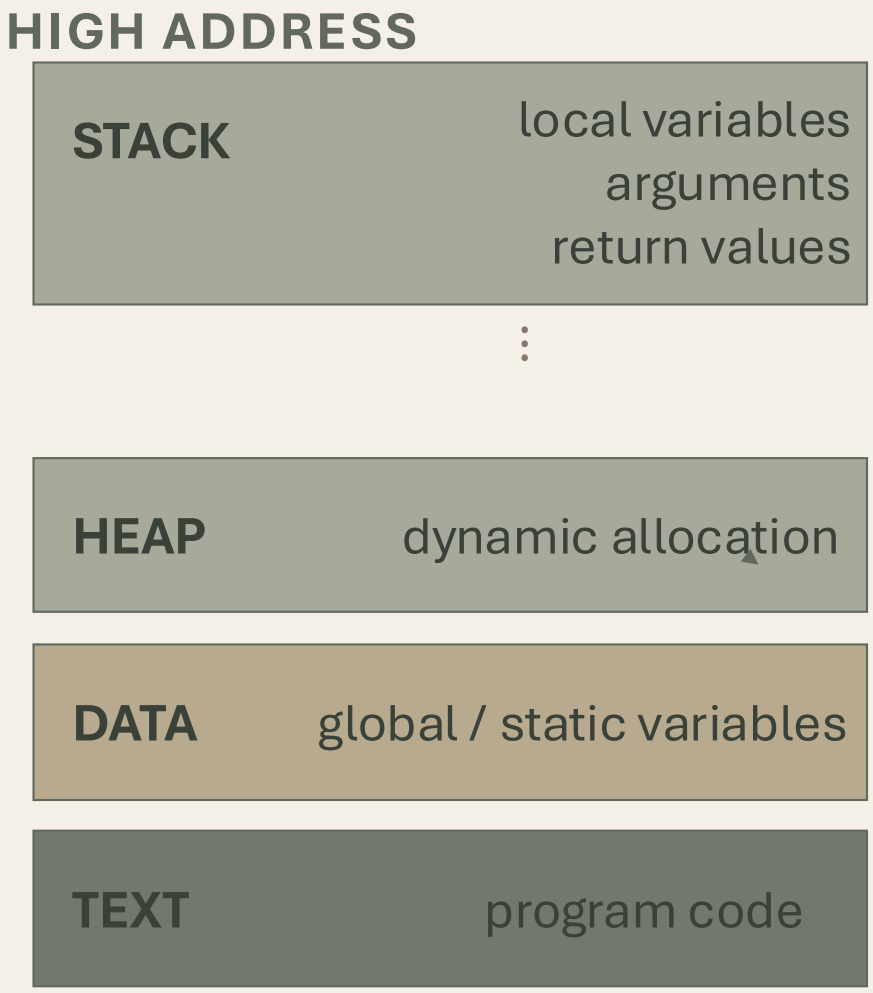
A PROCESS INCLUDES

Process Representation

Memory	Program code
	Data
Resource in use	I/O in use
	Memory
Execution state	Register set
	Process state

The OS does not manage “C files”. It manages running processes.

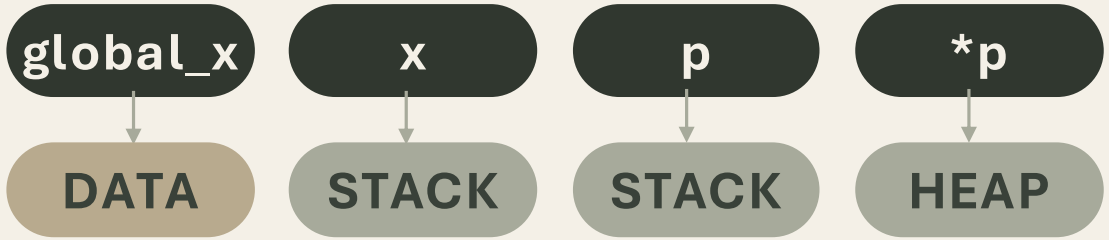
A Process Has an Address Space



SOURCE

```
int global_x; //global variable

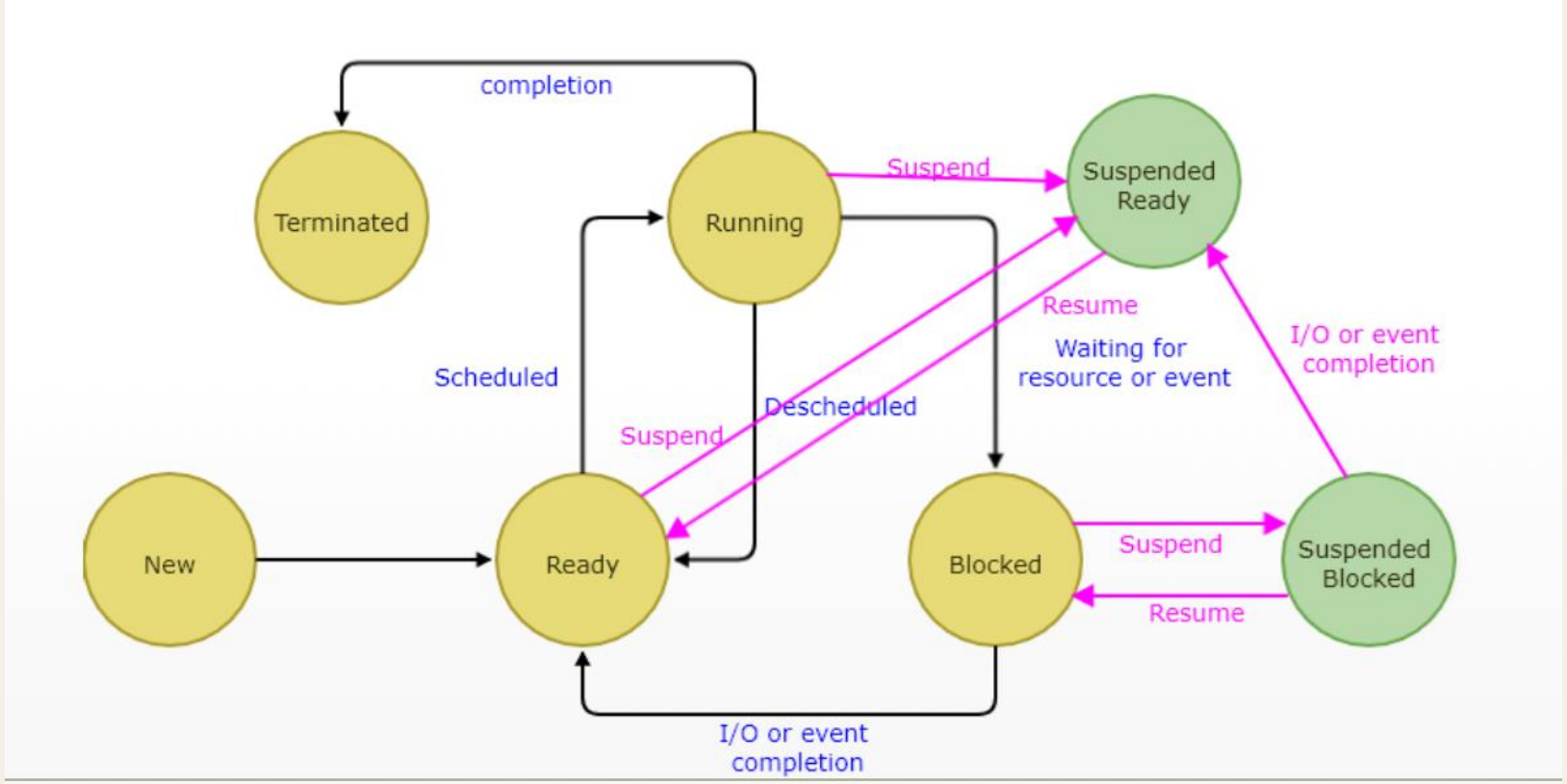
int main(void)
{
    int x = 10;
    int *p = malloc(sizeof(*p));
}
```



The pointer variable and the object it points to may live in different memory regions.

Process States and the PCB

PROCESS LIFECYCLE



OS RECORD

Process Control Block

PID
process state
program counter
CPU registers
scheduling info
memory info
parent / child
ptr to resources

To resume a stopped process, the OS must remember enough execution context.

UNIX Process API: fork(), exec(), wait()

fork()

```
pid_t pid = fork();
```

Creates a new process

Child initially duplicates parent

exec()

```
exec(...);
```

Replaces current process image

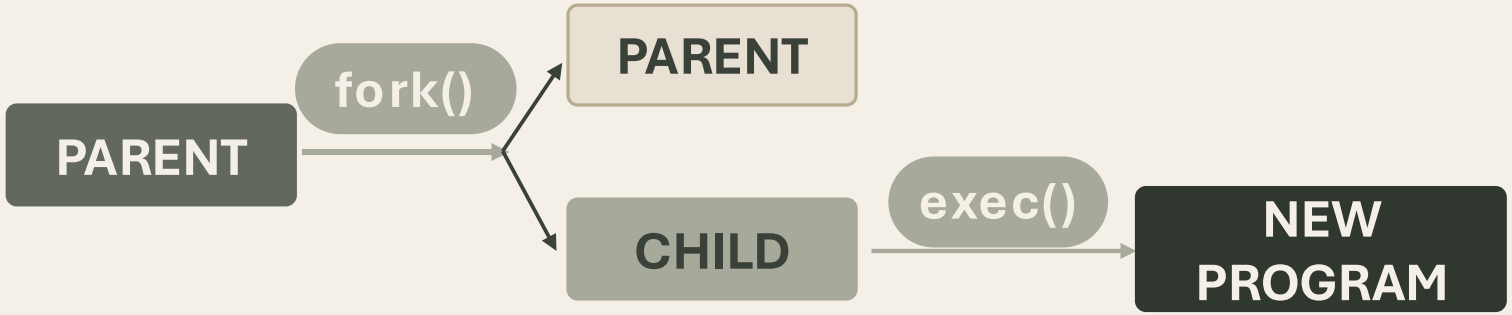
Does not create a process

wait()

```
wait(...);
waitpid(...);
```

Parent waits for child

Retrieves termination status



fork() creates a process. exec() changes what a process is running.

Signals: Notifying a Process of Events

WHAT IS A SIGNAL?

A signal is used in UNIX systems to notify a process that an event has occurred.

Each signal is identified by a symbolic name.

SIGINT	Ctrl-C is pressed
SIGCHLD	child finishes or terminates
SIGSEGV	invalid memory reference

EVENT → KERNEL → SIGNAL → PROCESS

TWO TYPES

SYNCHRONOUS

- Triggered by the current process itself Illegal memory access
- Example: invalid memory access → SIGSEGV

ASYNCHRONOUS

- Triggered by an external event or activity timer alarm
- Arrives independently of the target process's current instruction.

WHEN A SIGNAL ARRIVES

Catch — Run a predefined signal handler.

Ignore — Tell the OS not to perform the normal action.

Mask — Temporarily prevent delivery of that signal.

SIGKILL and SIGSTOP cannot be caught, blocked, or ignored.

Virtualizing the CPU

CPU TIMELINE



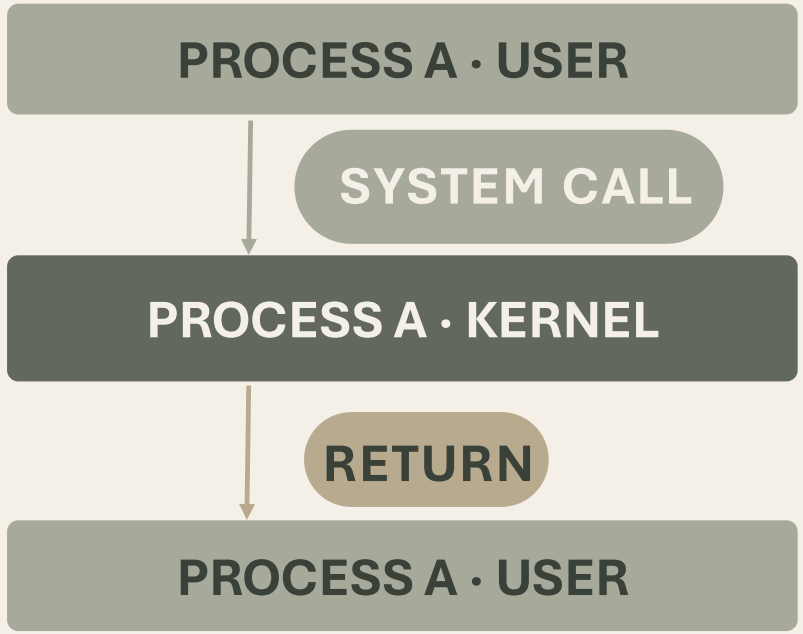
A CONTEXT SWITCH

01 **SUSPEND** current process → 02 **SAVE** execution context → 03 **CHOOSE** ready process → 04 **RESTORE** its context → 05 **CONTINUE** execution

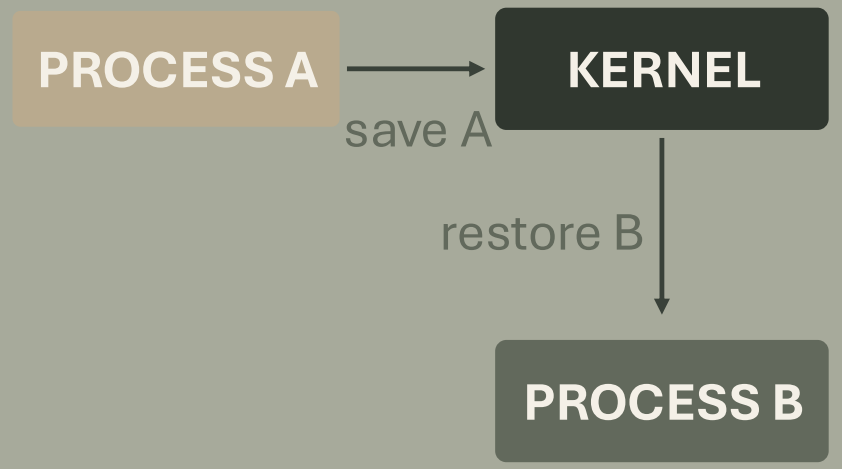
Each process behaves as if it has its own CPU, even though the hardware is shared.

Mode Switch ≠ Context Switch

SAME PROCESS
Mode Switch



DIFFERENT PROCESS
Context Switch



A system call causes a mode switch, but it does not necessarily cause a context switch.

Interrupts and Signals: Do Not Mix Them Up

CPU / KERNEL ATTENTION

Interrupt

Hardware or CPU event transfers control to the kernel.

- Timer interrupt
- Keyboard / device interrupt
- CPU exception

PROCESS NOTIFICATION

Signal

Kernel notifies a process that an event occurred.

- SIGINT — Ctrl-C
- SIGSEGV — invalid memory
- SIGCHLD — child changed state



Interrupts and signals are related ideas, but they are not the same mechanism.

Why Are We Using Linux?

WHY LINUX

A systems workbench

- Widely used for systems programming
- Open source
- Course grading platform (workbench2) is classic linux distribution -- Ubuntu

GRADING ENVIRONMENT = FINAL AUTHORITY

Linus Torvalds



Wrote the first version of



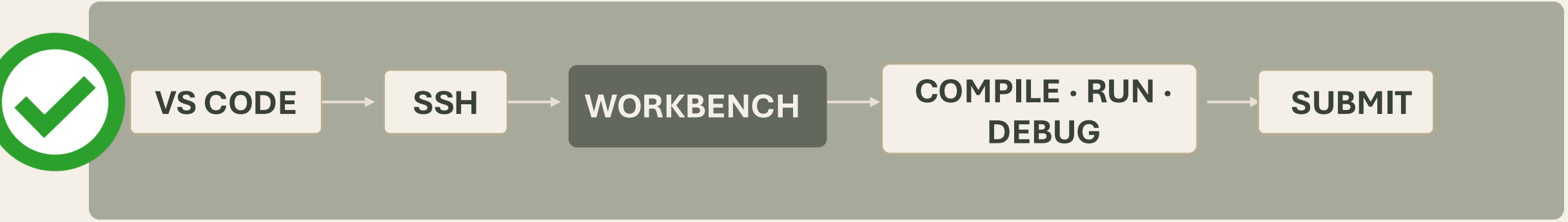
Linux

Has a popular distribution

Ubuntu

Recommended Development Workflow

REMOTE DEVELOPMENT



LOCAL DEVELOPMENT



Always test the final program in the grading environment.

Connection to Workbench2

Linux Commands You Will Actually Use

FILES

Navigate · inspect · manage

```
pwd     ls -l
cd      mkdir
cp      mv
rm      cat
```

PROCESSES & HELP

Observe · control · learn

```
man     ps
pstree  top
kill

ps -ef  pstree -p
```

```
man 2 open
man 3 printf
```

SECTION 2 system calls

SECTION 3 library functions

Pipes, Redirection, and Background Processes

PIPE
Connect commands

```
ps -ef | grep ssh
```

output → input

REDIRECTION
Send output

```
./program > output.txt
```

create / overwrite

BACKGROUND
Keep working

```
./program &
```

run without blocking shell

COMBINE THEM

```
./program > log.txt 2>&1 &
```

2>&1 sends standard error to the same destination as standard output.

Coding Example

Anatomy of a C Program

hello.c

```
#include <stdio.h>

int main(void)
{
    printf("Hello, COMP3230!\n");
    return 0;
}
```

#include declarations from a header

main() program entry point

{ … } a block

; end of a statement

return 0 successful termination

```
gcc hello.c -o hello
./hello
```

Compile C Properly

RECOMMENDED

```
gcc -Wall -Wextra -g program.c -o program
```

-Wall
common warnings

-Wextra
additional warnings

-g
debug information

-o
output filename

COMPILER ERROR
cannot compile

COMPILER WARNING
may compile · may be wrong

RUNTIME ERROR
fails while executing

In C, “it compiled” does not mean “it is correct.”

What Does a Makefile Solve?

WITHOUT MAKE

Repeat the whole command

```
gcc main.c parser.c util.c -o program
```

WITH MAKE

Describe dependencies once

```
program: main.o parser.o util.o
    gcc main.o parser.o util.o -o program
```

make



make is not a compiler. It decides which compilation commands should run.

```
make
make clean
```

Types You Will See Often

COMMON DECLARATIONS

```
char c;
int x;
long n;

size_t size;
ssize_t result;
pid_t pid;
```

DO NOT GUESS TYPE SIZES

```
sizeof(x)
sizeof(int)
sizeof(*ptr)
```

PREFERRED

```
int *a = malloc(
    10 * sizeof(*a));
```

AVOID

```
int *a = malloc(10 * 4);
```

Let the compiler tell you how large a type is.

printf(): Your Simplest Debugger

```
int x = 10;
double y = 3.14;
char c = 'A';
char *s = "hello";

printf("x = %d\n", x);
printf("y = %f\n", y);
printf("c = %c\n", c);
printf("s = %s\n", s);
```

FORMAT MAP

Specifier ↔ type

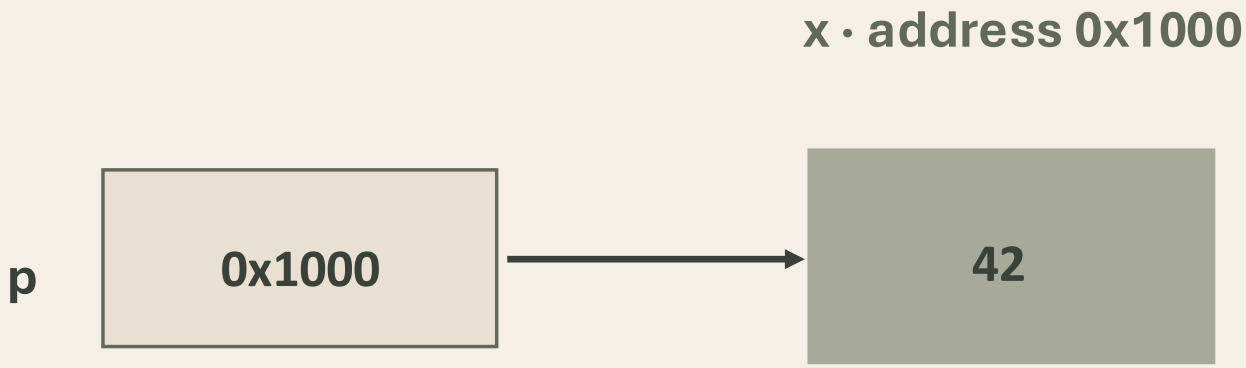
%d	int
%ld	long
%f	floating-point
%c	char
%s	char *
%p	pointer
%zu	size_t

```
printf("p = %p\n", (void *)p);
```

The format specifier must match the argument type.

Pointer: Think “Address”

```
int x = 42;
int *p = &x;
```



int *p pointer to int

&x address of x

***p** value at that address

```
*p = 100;
printf("%d\n", x);   // 100
```

A pointer is a variable whose value is an address.

NULL and Invalid Pointers

KNOWN EMPTY
NULL pointer

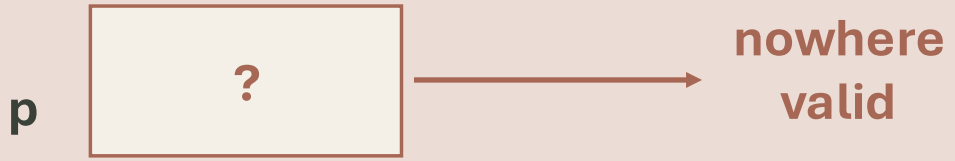
```
int *p = NULL;

if (p != NULL) {
    printf("%d\n", *p);
}
```

```
*p = 10;   // invalid
```

UNKNOWN GARBAGE
Uninitialized pointer

```
int *p;
*p = 10;
```



SEGMENTATION FAULT

> **Declaring a pointer does not allocate memory.**

Arrays and Pointers

```
int a[4] = {10, 20, 30, 40};
```

a[i] == *(a + i)

```
printf("%d\n", a[2]);
printf("%d\n", *(a + 2));
```

CONTIGUOUS MEMORY

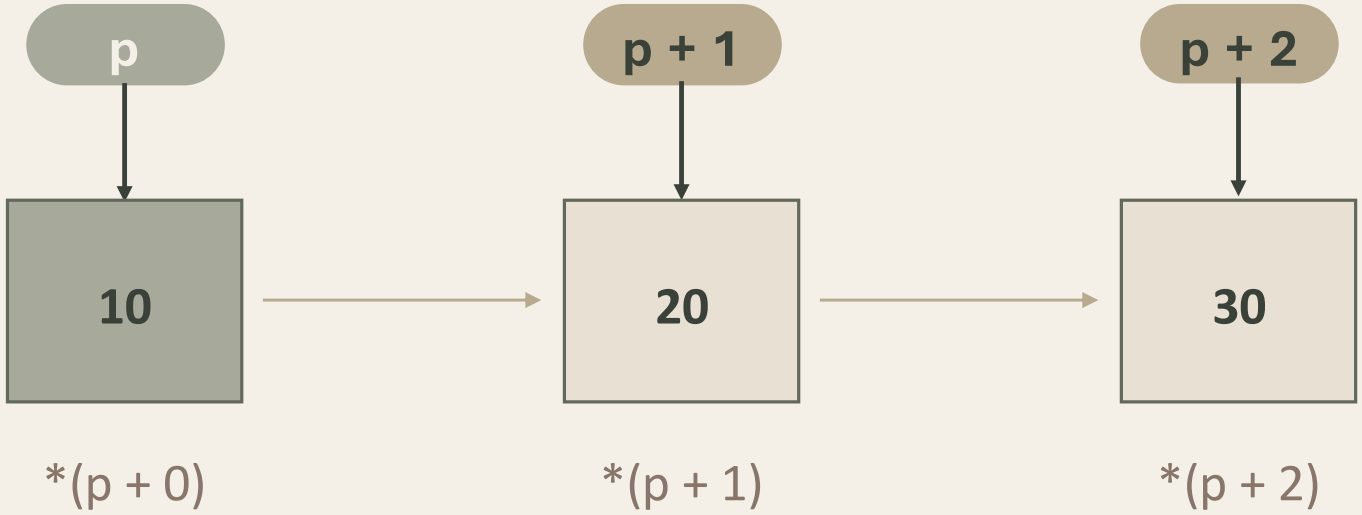


BOTH PRINT 30

Arrays and pointers are closely related in C, but they are not identical concepts.

Pointer Arithmetic

```
int a[] = {10, 20, 30};
int *p = a;
```



TYPE MATTERS

p + 1

moves by

sizeof(int)

not one byte

Pointer arithmetic advances in units of the pointed-to type.

C Passes Arguments by Value

COPIES ONLY

swap(int x, int y)

```
void swap(int x, int y)
{
    int tmp = x;
    x = y;
    y = tmp;
}

swap(a, b);   // unchanged
```

a = 1 b = 2 → copies

ADDRESSES

swap(int *x, int *y)

```
void swap(int *x, int *y)
{
    int tmp = *x;
    *x = *y;
    *y = tmp;
}

swap(&a, &b);
```

a = 2 b = 1 ← modified

Passing an address allows a function to modify the caller's object.

Why Do So Many C Functions Take Pointers?

POSIX API

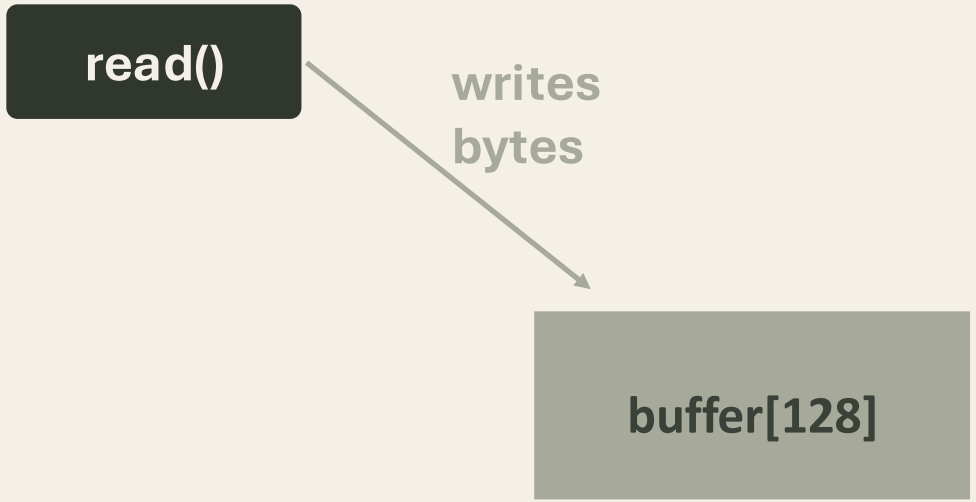
```
ssize_t read(int fd, void *buf, size_t count);
```

Why is buf a pointer?

Because read() must write into memory owned by the caller.

```
char buffer[128];

ssize_t n = read(fd, buffer, sizeof(buffer));
```



Pointers let functions access and modify memory outside their own local variables.

C Strings Are Character Arrays

```
char s[] = "hello";
```



strlen("hello") = 5

storage needed = 6 bytes

#INCLUDE <STRING.H>

Common operations

String operations

- strlen — get string length
- strcmp — compare two strings
- strcpy — copy one string to another
- strcat — append one string to another

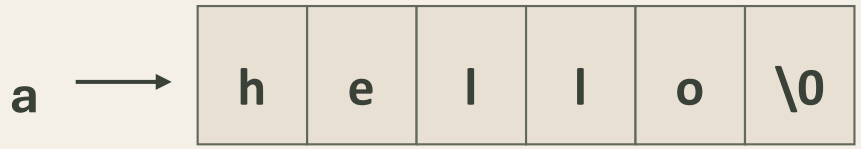
Memory operations

- memcpy — copy a block of raw bytes
- memset — fill a block of memory with a byte value

char * vs. char []

WRITABLE ARRAY

char a[] = "hello"




```
a[0] = 'H';   // OK
```

STRING LITERAL

char *b = "hello"




```
b[0] = 'H';   // do not rely on this
```

```
char buffer[256];
```

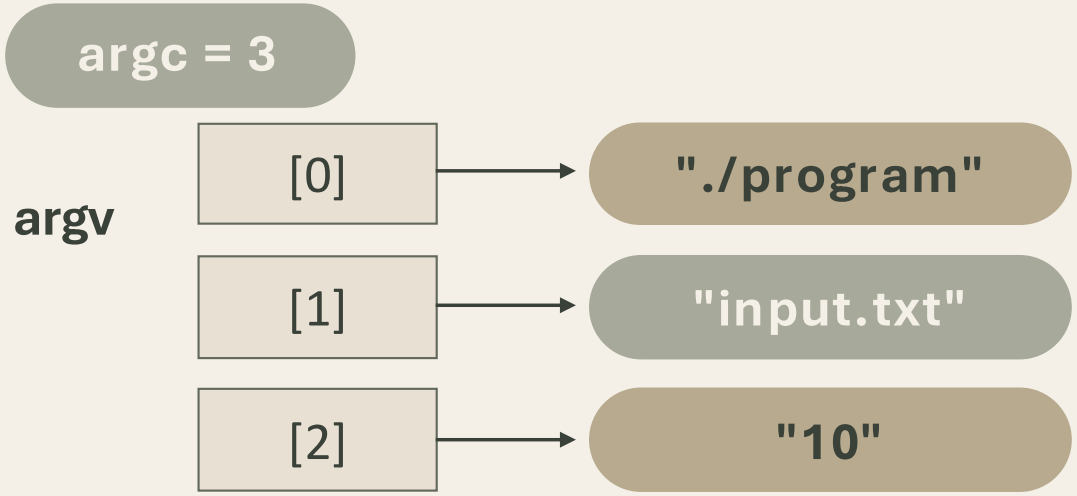
Need a modifiable string buffer? Allocate an array or writable memory.

Command-Line Arguments

```
int main(int argc, char *argv[])
```

SHELL

```
./program input.txt 10
```




```
int main(int argc, char **argv)
{
    for (int i = 0; i < argc; i++) {
        printf("argv[%d] = %s\n", i, argv[i]);
    }
    return 0;
}
```

Every command-line argument is initially a string.

Coding Example

struct: Group Related State Together

```
struct process {
    int pid;
    int state;
    char name[32];
};

struct process p;
p.pid = 123;

struct process *ptr = &p;
ptr->pid = 456;
```

IN MEMORY

process p

pid	456
state	…
name[32]	…

ptr->pid

≡ (*ptr).pid

OS code models processes, files, sockets, memory regions, and page-table entries with structs.

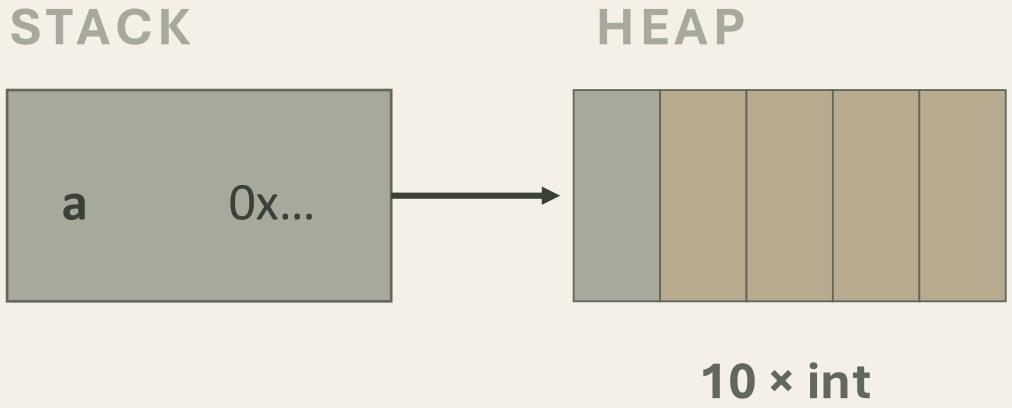
struct

Dynamic Memory: malloc() and free()

```
#include <stdlib.h>

int *a = malloc(10 * sizeof(*a));

if (a == NULL) {
    // allocation failed
}
```




```
free(a);
a = NULL;
```

Every allocation needs a clear lifetime and ownership story.

malloc(), calloc(), realloc(), free()

01
malloc()

```
p = malloc(n * sizeof(*p));
```

allocates · contents unspecified

02
calloc()

```
p = calloc(n, sizeof(*p));
```

allocates · zero-initializes

03
realloc()

```
tmp = realloc(p, new_size);
```

resizes · may move to new address

04
free()

```
free(p);
```

releases allocation

Four Memory Bugs You Must Recognize

01 UNINITIALIZED POINTER

```
int *p;
*p = 10;
```

02 MEMORY LEAK

```
p = malloc(100);
// no free(p)
```

03 USE AFTER FREE

```
free(p);
*p = 10;
```

04 OUT OF BOUNDS

```
int a[10];
a[10] = 5;
```

POSSIBLE SYMPTOMS

C does not automatically protect you from invalid memory access.

The C Preprocessor and Macros

```
#include <stdio.h>
#include <stdlib.h>

#define BUFFER_SIZE 1024
```

```
#ifdef DEBUG
printf("x = %d\n", x);
#endif

gcc -DDEBUG program.c -o program
```



MACRO PITFALL

BAD

```
#define SQUARE(x) x * x
```

BETTER

```
#define SQUARE(x) ((x) * (x))
```

> **The preprocessor transforms source before compilation.**

Debugging C: Do Not Guess

```
gcc -Wall -Wextra -g program.c -o program
```

DEBUGGER ACTIONS -gdb

break · run · inspect

```
break   run   next   step
continue   print   backtrace
```

VS CODE

The same mental model

Breakpoint · Step Over · Step Into Continue · Variables · Watch · Call Stack

REPRODUCE → **FIND FIRST WRONG STATE** → **INSPECT** → **UNDERSTAND** → **FIX CAUSE**

Fix the cause, not the symptom.

When You See “Segmentation Fault”

Do not immediately start changing random code.

- ☐ Dereferenced NULL?
- ☐ Used an uninitialized pointer?
- ☐ Accessed outside an array?
- ☐ Wrote beyond a string buffer?
- ☐ Used memory after free()?
- ☐ Passed the wrong pointer?

```
gdb ./program

run
bt
frame 0
print variable
```



Segmentation fault is a symptom. Locate the invalid memory access.

What You Should Be Comfortable With After Today

OS CONCEPTS
READY TO USE

- ☐ User vs. kernel mode
- ☐ Process & address space
- ☐ Process states & PCB
- ☐ System call
- ☐ Context switch
- ☐ Interrupt
- ☐ Signal

LINUX
READY TO USE

- ☐ SSH / Workbench
- ☐ man
- ☐ ps / pstree / top
- ☐ Pipe & redirection
- ☐ gcc
- ☐ make
- ☐ git

C
READY TO CODE

- ☐ **pointers: * and &**
- ☐ **arrays & strings**
- ☐ **argc / argv**
- ☐ **struct and ->**
- ☐ **malloc / free**
- ☐ **parameter passing**
- ☐ **function pointers**
- ☐ **printf**
- ☐ **open / read / write**